



AI-ASSISTED JOB POSTING PORTAL AND SKILL GAP TRACKER

Complete Project Explanation and Technical Guide

Project Member	IJ Maayav (44110430)
Guide	Dr. N. UmaSankari
Project Name	Vortex

Full-stack MERN application with grounded AI career guidance

Prepared from the implemented repository and deployed system | September 2026

Project at a Glance

Vortex combines a student job portal with an evidence-based career-readiness workspace. Students can discover opportunities, apply for jobs, upload a resume, connect optional GitHub and LeetCode evidence, inspect extracted skills, compare their profile against a selected role, and follow a personalized study roadmap. Placement administrators can manage job openings, review candidate evidence, and move applications through a structured pipeline.

The system is designed around a practical problem: a student may know the technologies listed on a job description but still lack a clear answer to three questions: which skills are actually demonstrated, which requirements remain weak, and what should be learned first. Vortex answers these questions through a controlled pipeline that combines text AI, embeddings, a curated role ontology, deterministic scoring, verified resources, and a grounded assistant.

MERN React + Node.js + Express + MongoDB	AI Groq text generation and Gemini embeddings	RBAC Student and administrator workflows	Live Cloudflare Pages, Vercel, and Atlas
--	---	--	--

Current catalog: 21 target roles, 149 ontology skills, 307 curated resource records, and 180 searchable keyword names are available in the deployed seed.

Core outcome: the application does not present readiness as a hiring guarantee. It presents a transparent, evidence-based decision-support signal and converts verified gaps into actionable preparation steps.

Keywords

- Job Portal
- Skill Extraction
- Readiness Scoring
- Study Roadmap
- JWT
- MongoDB
- Grounded AI

Contents

1. Introduction and problem definition	12. REST API design
2. Objectives, scope, and requirements	13. Frontend design and user experience
3. Technology stack	14. Database design and relationships
4. System architecture	15. Deployment architecture
5. Major modules and data flow	16. Deployment guide and runbook
6. Authentication, authorization, and security	17. Testing and verification
7. Job portal and application workflow	18. Security, limitations, and future work
8. Profile ingestion and skill extraction	19. Conclusion and viva summary
9. Analytics, matching, and scoring engine	Appendix A. Repository structure
10. Study roadmap and resource curation	Appendix B. API examples
11. Grounded AI Assistant	Appendix C. Deployment checklist

Document Control

Item	Value
Product name	Vortex
Repository	Job-Posting-Portal, branch main
Source of truth	Implemented backend, frontend, tests, seed data, and deployment configuration
Report purpose	Academic explanation, technical reference, viva preparation, and deployment handover
Security note	Secrets, API keys, database credentials, and passwords are intentionally excluded

Introduction and Problem Definition

1.1 Background

Campus placement systems commonly separate opportunity discovery from preparation. A student may find a job posting, submit an application, and receive a status update, but the system often does not explain how the student's demonstrated profile compares with the role requirements. This creates a gap between applying and improving.

Vortex closes this gap by combining job discovery, application tracking, profile evidence, role-specific analytics, and preparation guidance in one authenticated workspace. It is intended for students, placement teams, and academic demonstrations where explainability and secure ownership are important.

1.2 Problem Statement

Problem: students need an understandable and actionable method to assess their readiness for a target job, while placement administrators need a controlled way to manage openings and review candidates without exposing private profile data.

1.3 Proposed Solution

Vortex accepts a resume and optional public coding profiles, extracts structured skills with supporting evidence, validates and normalizes those skills, compares them against a curated target-role ontology, calculates readiness categories, and produces a prioritized roadmap backed by verified learning resources. The same platform supports job search, wishlist, applications, notifications, and administrator review.

1.4 Design Principles

Evidence before claims

Extracted skills carry source and evidence excerpts. Confidence is derived by application rules rather than blindly accepted from the model.

Server-side ownership

User identity, applicant ownership, report access, and administrator permissions are derived from verified JWT claims and backend middleware.

Deterministic decision layer

AI helps structure and explain information, while scoring, thresholds, priorities, and resource selection remain controlled by application code.

Actionable output

The final result is not only a number. It includes gaps, priorities, learning resources, practice suggestions, and a completion state.

Objectives, Scope, and Requirements

2.1 Objectives

1. Provide a secure job board for authenticated students and administrators.
2. Support job search by skill, experience, city, and pagination.
3. Allow students to apply, save jobs, review application stages, and receive notifications.
4. Extract technical skills from resumes and optional public developer profiles.
5. Compare candidate evidence with a role-specific skill ontology.
6. Generate a transparent readiness score, gap list, and personalized study roadmap.
7. Give administrators a candidate review workspace with application filters and status transitions.
8. Keep AI responses grounded in stored reports and prevent invented candidate claims.

2.2 Functional Requirements

ID	Requirement	Implemented behavior
FR-01	Authentication	Register and login with bcrypt password hashing and JWT sessions.
FR-02	Role access	Public registration creates students; admin capabilities are protected by server-side role middleware.
FR-03	Job management	Students search jobs; administrators create, edit, close, and delete postings.
FR-04	Applications	One application per student/job pair, status history, optional cover letter and resume.
FR-05	Profile analysis	PDF validation, text extraction, optional GitHub/LeetCode collection, and role selection.
FR-06	Readiness report	Score, strong areas, developing areas, gaps, study plan, and report history.
FR-07	Roadmap progress	Per-role roadmap selection and collapsible study-plan cards with completion toggles.
FR-08	AI guidance	Assistant answers only from the latest owned report context.
FR-09	Catalog expansion	Dynamic target roles, canonical skill keywords, curated documentation, courses, practice links, and specific tutorial videos.
FR-10	Resume plausibility	Readable PDF extraction plus a server-side heuristic rejects course handouts and unrelated PDFs before AI analysis.

2.3 Non-functional Requirements

Security

Secrets remain server-side; sensitive resume text is not returned; upload magic bytes and size are checked; ownership is enforced.

Reliability

Central error handling, validation, rate limiting, idempotent analysis jobs, cached embeddings, and controlled provider fallbacks.

Usability

Responsive React interface, clear loading/error states, accessible controls, dashboard summaries, and separate student/admin navigation.

Maintainability

Separated routes, controllers, services, models, tests, seed scripts, documentation, and deployment configuration.

Technology Stack

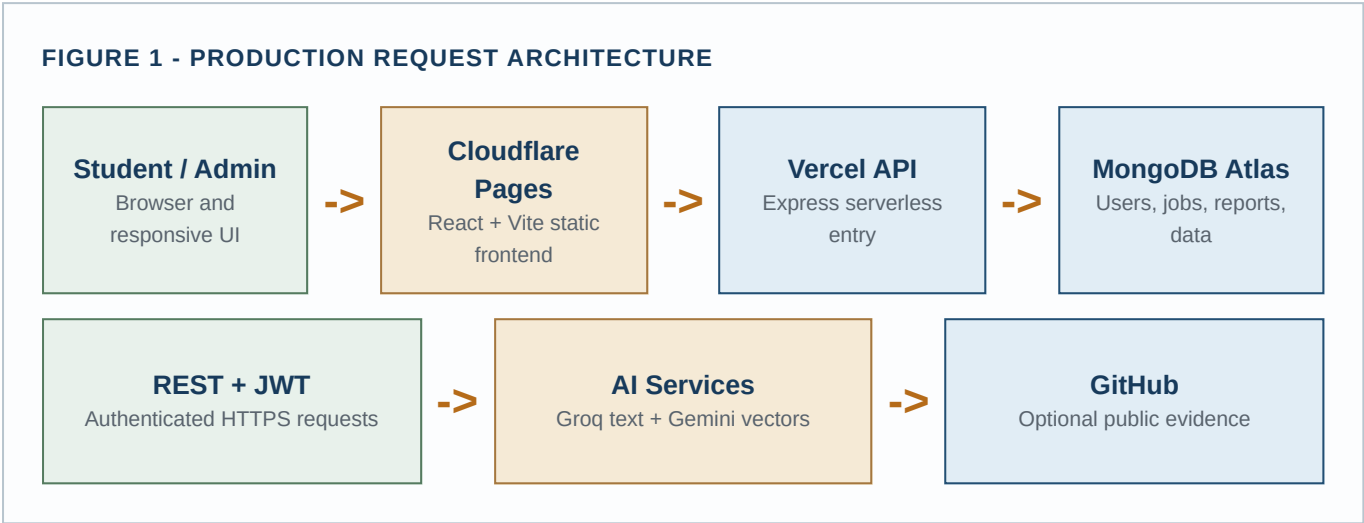
Technology	Purpose in Vortex
React 19 + Vite	Interactive frontend, protected routes, forms, dashboards, job browsing, analysis flow, and AI Assistant.
React Router	Client-side routing for landing, authentication, jobs, dashboard, analysis, applications, and admin pages.
Motion + Lenis	Reduced, intentional page transitions, scroll reveals, and interaction feedback.
Recharts	Readiness history visualization and progress trend presentation.
Node.js + Express 5	REST API, middleware, controllers, error handling, authentication, and business services.
Mongoose 9	MongoDB models, validation, relationships, indexes, and aggregation-backed dashboard queries.
Zod	Strict request validation and structured AI response validation.
MongoDB Atlas	Cloud persistence for users, jobs, applications, profile submissions, reports, ontology, and resource catalog.
Groq	Text generation for structured skill extraction, study-plan enrichment, explanations, and assistant responses.
Gemini	Skill embeddings and semantic vector storage using a pinned model/version.
GitHub API	Best-effort collection of public repositories, languages, topics, and bounded README evidence.
unpdf + file-type + Multer	Server-side PDF extraction, magic-byte validation, and bounded multipart uploads.
Vitest + Supertest	Backend integration tests and frontend component tests.

3.1 Versioned Runtime Baseline

Frontend: React 19, Vite 8, React Router 7, Axios 1.x, Recharts 3.x Backend: Node.js 22.x or later, Express 5, Mongoose 9, Zod 4 Database: MongoDB 7-compatible deployment AI text provider: Groq, model selected through environment configuration Embedding provider: Gemini, model gemini-embedding-2, version 2026-09 Test tooling: Vitest 5, Supertest 7, Testing Library

Separation of concerns: the browser knows the public API base URL, but never connects directly to MongoDB and never receives Groq, Gemini, GitHub, JWT-signing, or database credentials.

Overall System Architecture



The frontend is a client of the API. Data ownership, validation, AI calls, and database access remain in the backend.

4.1 Layer Responsibilities

Client layer	Pages, navigation, protected routes, forms, loading states, dashboards, charts, application controls, and the AI Assistant view.
API layer	Express routes, authentication middleware, role checks, upload handling, request validation, rate limits, and centralized errors.
Domain layer	Profile processing, skill normalization, ontology lookup, deterministic scoring, study-plan generation, application transitions, and notifications.
Persistence layer	Mongoose models and indexes for users, profile submissions, extracted skills, reports, jobs, applications, wishlists, notifications, ontology, and resources.
Provider layer	Groq text generation, Gemini embeddings, GitHub public-data collection, and verified resource catalog hydration.

4.2 Request Lifecycle

Browser -> Axios client -> CORS and rate limiter -> JWT middleware -> Zod/controller validation
-> domain service -> Mongoose query / AI provider -> safe response -> React state update

Major Modules and Data Flow

Module 1 - Data Space

Stores users, roles, jobs, saved jobs, applications, status history, profile submissions, extracted skills, reports, notifications, ontology entries, and verified resources.

Module 2 - Analytics

Reads resume and public profile evidence, extracts structured skills, normalizes names, caches vectors, and compares candidate evidence with the selected role.

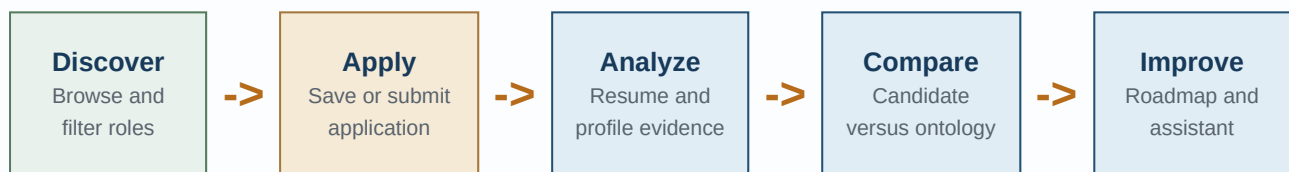
Module 3 - Scoring Engine

Calculates ATS/readiness output, categorizes skills, ranks gaps, hydrates resources, and creates a durable report.

Module 4 - AI Assistant

Loads trusted context from the latest owned report and generates grounded explanations without inventing skills, experience, or scores.

FIGURE 2 - END-TO-END PRODUCT FLOW



5.1 Student Journey

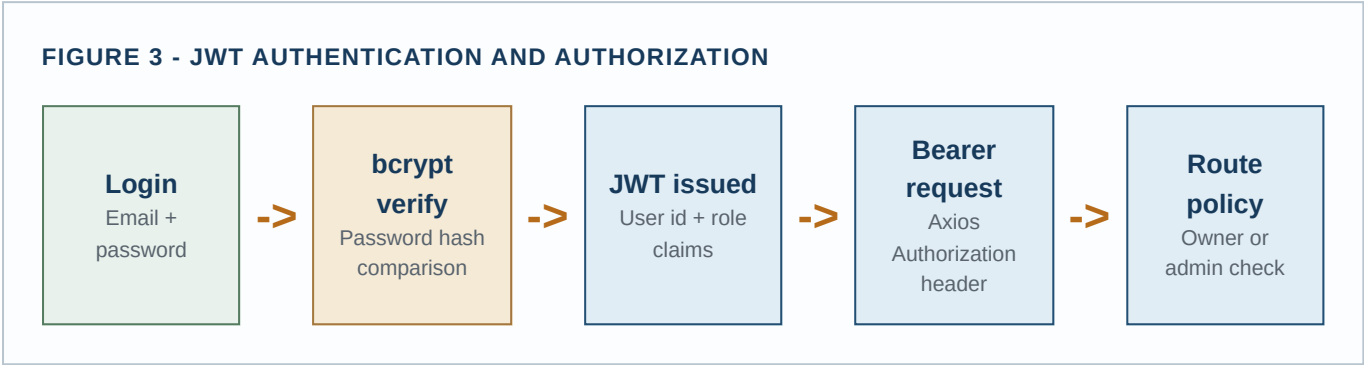
1. Register or log in as a student.
2. Browse jobs and filter by canonical skills, experience, or city.
3. Save a job or submit an application with optional contact email, cover letter, and resume.
4. Start New Analysis for a target role and upload a PDF resume.
5. Review extracted skills and evidence before scoring.
6. Open the dashboard to inspect score, role roadmaps, application preparation, and history.
7. Expand a roadmap, complete study items, and ask the grounded Assistant for next steps.

5.2 Administrator Journey

Administrators log in through the same JWT system, manage job postings, view applications, filter candidate records, inspect evaluation context, and transition applications through Applied, Under Review, Shortlisted, Interview Scheduled, Rejected, or Selected.

Authentication, Authorization, and Security

6.1 Authentication Flow



6.2 Access Matrix

Action	Student	Admin	Enforcement
Register / login	Yes	Yes	Auth controller; public registration always creates student.
View and filter jobs	Yes	Yes	Authenticated job route.
Create or edit jobs	No	Yes	<code>requireRole('admin')</code> .
Apply / save jobs	Yes	Optional	Student route derives applicant from JWT.
Read own reports	Yes	Own/admin scoped access	Ownership middleware and report access.
View all applications	No	Yes	Admin-only routes.
Run New Analysis	Yes	Optional	Protected student flow and owned submission.

6.3 Security Controls

- [] Passwords are stored using bcrypt hashes and never returned.
- [] JWT secrets, MongoDB credentials, Groq keys, Gemini keys, and GitHub tokens remain in server environment variables.
- [] Resume files require PDF magic bytes, are size-limited, and are stored outside the public frontend.
- [] Resume text is sensitive: it is not returned by profile APIs and is not written to logs.
- [] Student ownership is checked for profiles, reports, applications, wishlists, and notifications.
- [] Rate limits protect authentication, analysis, and the general API.
- [] Validation rejects unknown or malformed fields before domain logic.

Job Portal and Application Workflow

7.1 Job Search

Job records contain title, company, canonical skills, experience level, city, description, status, and server-derived creator ownership. A normalized search projection supports case-insensitive skill search and city matching. Skill synonyms such as ReactJS, NodeJS, and UI/UX are normalized explicitly while unrelated categories remain separate.

Filter	Semantics
Skills	Comma-separated ANY-match, normalized and case-insensitive.
Experience	Returns jobs whose required level is less than or equal to the query.
City	Case-insensitive exact match after trimming.
Pagination	Default page 1, limit 20, maximum 50.
Sorting	Newest postings first.

7.2 Application State Machine



7.3 Application Integrity

- A compound unique index prevents duplicate applications for the same student and job.
- The applicant is derived from the verified JWT, never trusted from the request body.
- Every status transition records a history entry and administrator identity when applicable.
- Students see only their own applications; administrators can filter and review all applications.
- Notifications are created for application confirmation and administrator status changes.

Profile Ingestion and Skill Extraction

8.1 Inputs

Required

PDF resume and target role selected from the live role ontology.

Optional

GitHub username or URL and LeetCode profile identifier. Public profile failure degrades gracefully.

8.2 Ingestion Pipeline

FIGURE 5 - PROFILE INGESTION PIPELINE



8.3 Structured Skill Record

Each extracted skill is stored with a display name, category, source list, evidence excerpts, and proficiency signals. The application merges casing duplicates and derives confidence using fixed rules: high confidence requires evidence from multiple sources, medium confidence requires one source with evidence, and low confidence represents a weak or bare mention.

```

{
  "name": "React",
  "category": "frontend_framework",
  "sources": ["resume", "github"],
  "evidence": [{ "source": "resume", "text": "Built a dashboard using React" }],
  "proficiency_signals": {
    "projects_count": 2,
    "has_production_usage": true,
    "mentions_depth": "high"
  },
  "confidence": "high"
}
  
```

Important matching rule: related-looking technologies are not automatically interchangeable. React does not prove PyTorch; JavaScript does not prove Python; Node.js does not prove Machine Learning; UI/UX does not automatically prove frontend engineering. Upload validation also screens PDFs for resume signals before AI runs.

Analytics, Matching, and Scoring Engine

9.1 Role Ontology

The ontology is the controlled definition of role expectations. Each skill includes a canonical name, category, embedding model/version, vector, and one or more role weights. The role list is derived from the stored ontology rather than hard-coded in the upload form. The current seed contains 21 roles: SDE, ML Engineer, AI Engineer, Frontend Developer, Mobile App Developer, Data Analyst, Site Reliability Engineer, Blockchain Developer, Game Developer, UI/UX Designer, Network Engineer, Database Administrator, Business Analyst, Backend Developer, Cloud Engineer, Cybersecurity Analyst, Data Engineer, Data Scientist, DevOps Engineer, Full-Stack Developer, and QA/Test Engineer.

9.2 Candidate Vector Cache

Candidate skill vectors are generated in batches and cached on the extracted profile together with the configured embedding model and version. Analysis reuses a compatible cache and re-embeds only when the cache is missing or the model/version changes. Stored ontology vectors are checked for model/version and dimension compatibility before scoring.

9.3 Deterministic Formula

$$\text{score} = \text{round}(100 \times \text{sum}(\text{weight}_i \times \text{match}_i) / \text{sum}(\text{weight}_i))$$

Each role skill receives a normalized match value. Explicit canonical names and safe aliases are considered first, and category gates prevent unrelated categories from satisfying one another. Candidate evidence and proficiency can cap a match when the supporting signal is weak.

Category	Threshold	Meaning
Strong	80% or above	Skill is strongly aligned with the role requirement.
Developing	60% to 79%	Skill is present but needs depth, evidence, or practice.
Gap	Below 60%	Skill should be considered for the prioritized roadmap.

9.4 Gap Priority

Gap priority is calculated from role importance and missing strength: $\text{raw} = \text{weight} \times (1 - \text{match})$. The raw values are normalized by the largest gap so the highest-impact missing skill receives priority 1.0. This provides a ranking without pretending that every gap has identical urgency.

```
const raw = gap.weight * (1 - gap.m);
priority = raw / maxRaw;
study_plan = gaps.sort((a, b) => b.priority - a.priority);
```

Study Roadmap and Resource Curation

10.1 Roadmap Structure

A completed report contains a study plan entry for each gap or selected developing area. The current catalog contains 307 curated records across 180 searchable skill names. Each entry can include the skill, priority, reason, learning objectives, practice problems, project recommendations, estimated effort, completion state, and verified resources.

Field	Purpose
skill	Canonical gap or developing skill name.
priority	Relative urgency derived from role weight and match deficit.
resources	Human-curated documentation, courses, videos, or practice sets.
done	Student-owned completion state toggled from the dashboard.
learningObjectives	Concrete knowledge outcomes when AI enrichment is available.
projectRecommendations	Practical implementation ideas connected to the gap.
estimatedEffortHours	Planning estimate for a focused learning block.

10.2 Dashboard Interaction

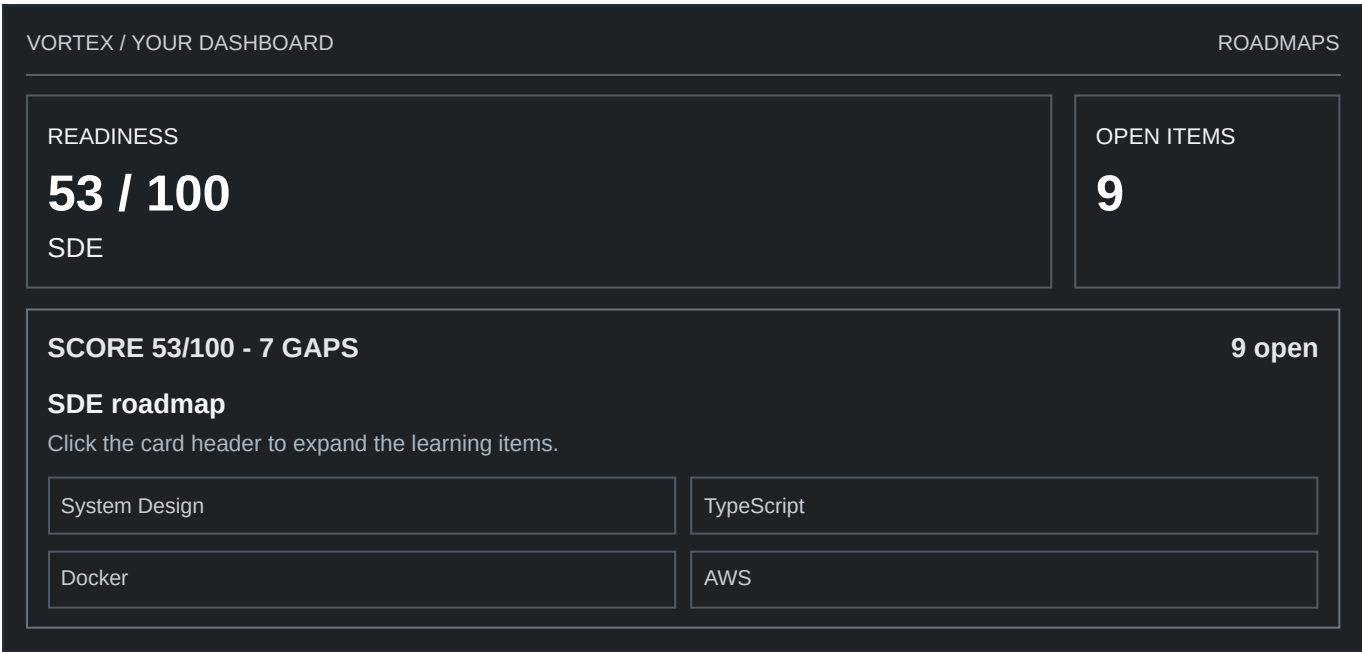


Figure 6 - Dashboard roadmap concept: role selection, collapsible roadmap summary, and actionable items.

10.3 Resource Safety

ResourceCatalog serves verified HTTP/HTTPS links, including specific video watch URLs; arbitrary model-generated links are not final resources. The dedicated GET /api/skills endpoint keeps filters synchronized.

Grounded AI Assistant

11.1 Context Boundary

The Assistant is not a general chatbot with unrestricted access to the database. The backend first loads the student's latest completed owned report and constructs a constrained context containing the target role, scores, demonstrated skills, gaps, evidence summaries, and study plan. The user's message is then sent to the configured text provider with grounding instructions.

FIGURE 7 - GROUNDED ASSISTANT FLOW



11.2 Allowed Questions

- Which gap should I learn first?
- Why is my readiness score low?
- What project can I build to practice this gap?
- Which skills are already supported by my evidence?
- How should I prepare for the selected role?

11.3 Guardrails

- [] Do not invent skills, projects, experience, certifications, or scores.
- [] Do not claim that unrelated technologies satisfy a requirement.
- [] Do not expose another student's analysis or private resume text.
- [] If no completed analysis exists, direct the student to New Analysis.
- [] Return a controlled error when provider configuration is unavailable.

Interpretation: the Assistant explains stored analysis; it does not replace the deterministic scoring layer or human evaluation.

REST API Design

The API uses a consistent JSON error shape, authenticated Bearer tokens, route-level ownership, and strict validation. The public health route and authentication routes are intentionally minimal; scoped data routes require a JWT.

Area	Representative endpoints	Access
Health	GET /api/health	Public
Auth	POST /api/auth/register POST /api/auth/login	Public
Roles	GET /api/roles	Authenticated
Skills	GET /api/skills	Authenticated; cached keyword catalog
Profile	POST /api/profile GET /api/profile/:id DELETE /api/profile/:id	Owner/admin
Analysis	POST /api/analyze GET /api/analyze/:id/status	Owner/admin
Reports	GET /api/report/:id GET /api/report/history GET /api/report/roadmap	Owner/admin as scoped
Study plan	PATCH /api/report/:id/study-plan/:itemId	Owner/admin
Jobs	GET /api/jobs POST /api/jobs PUT /api/jobs/:id DELETE /api/jobs/:id	Read authenticated; write admin
Applications	POST /api/applications GET /api/applications/me	Student / own data
Admin review	GET /api/admin/dashboard GET /api/admin/applications PATCH /api/admin/applications/:id/status	Admin only
Assistant	GET /api/assistant/context POST /api/assistant/chat	Authenticated, own context
Notifications	GET /api/notifications PATCH /api/notifications/:id/read	Authenticated, own data
Wishlist	GET /api/wishlist POST /api/wishlist DELETE /api/wishlist/:jobId	Student, own data

Frontend Design and User Experience

13.1 Main Routes

Route	Purpose	Protection
/	Public Vortex landing page.	Public
/login	Login and account entry.	Public
/jobs	Job search, filters, save, and apply.	Authenticated
/dashboard	Student readiness, roadmaps, applications, wishlist, and progress.	Authenticated
/analyze	Upload, review skills, analyze, and poll report state.	Student
/applications	Student application history and status.	Authenticated
/assistant	Grounded AI guidance.	Authenticated
/admin/jobs	Job CRUD.	Admin
/admin/applications	Candidate review dashboard and pipeline.	Admin

13.2 Dashboard Information Hierarchy

1. Readiness score for the latest report.
2. Role-based roadmap selector and collapsible roadmap header.
3. Preparation prompts based on skills in applied jobs.
4. Application count and link to the full application page.
5. Saved jobs with apply and unsave actions.
6. Historical readiness trend.

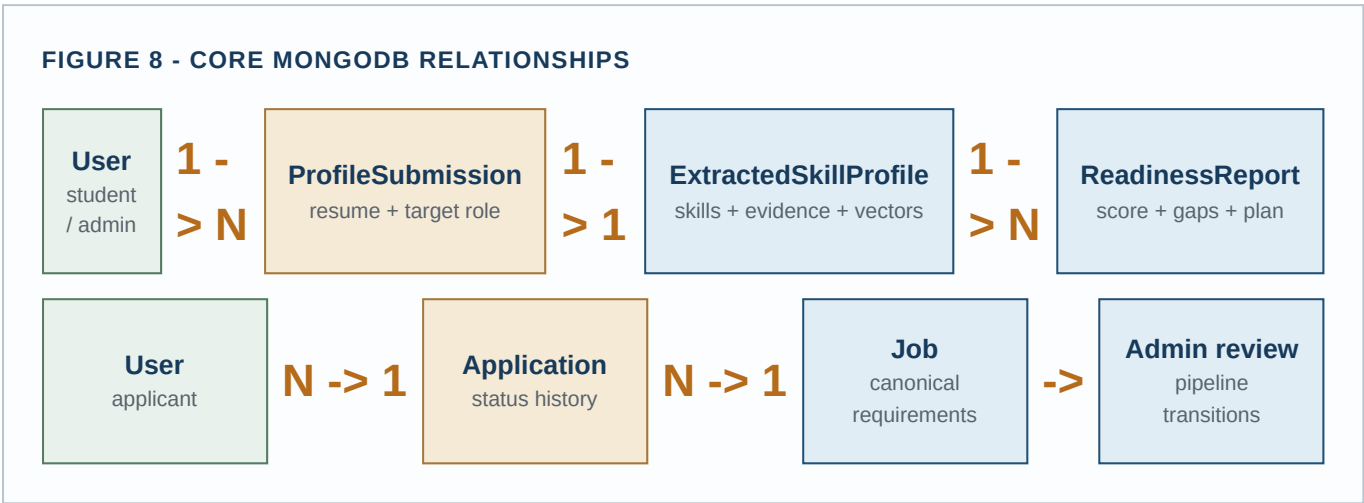
13.3 UX and Accessibility Decisions

- Loading, empty, error, and retry states are explicit in analysis and dashboard flows.
- Roadmap cards use an accessible expanded/collapsed control and preserve the selected role.
- Job filters and administrator job forms use a searchable multi-select keyword picker with custom skill entry.
- Forms use labels, validation messages, disabled states, and direct action wording.
- Responsive layout supports desktop and mobile widths without hiding core destinations.
- Theme preference persists through the application and remains readable in both modes.
- Motion is intentionally reduced when the user requests reduced motion.
- Route-level lazy loading and deferred chart loading reduce the initial JavaScript payload.

13.4 Loading Performance

Lazy routes and deferred Recharts reduce initial JavaScript from 969 kB to 466 kB; role caching and one-query roadmap hydration reduce request overhead.

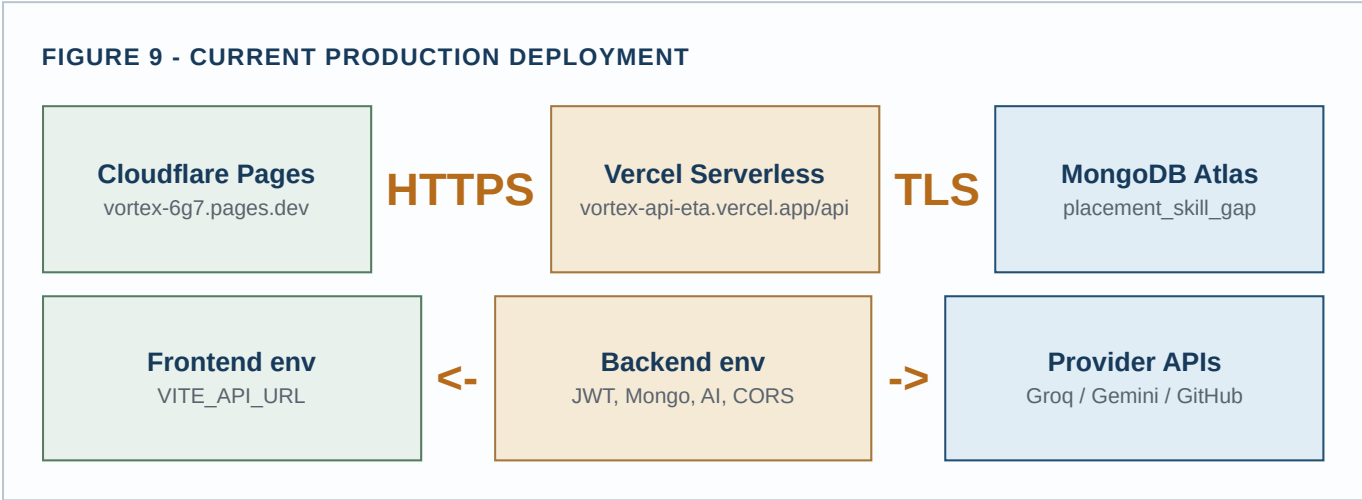
Database Design and Relationships



Collection	Key responsibility	Important protection/index
users	Identity, password hash, role.	Unique lowercase email; password excluded by default.
profiles submissions	Resume metadata, sensitive text, profile links, target role.	User index; ownership checks; text never returned.
extracted skill profiles	One extracted profile per submission.	Unique submission id; cached vector model/version.
readiness reports	Analysis lifecycle, score, gaps, roadmap, generation metadata.	Partial unique index for one active job per submission.
skill ontologies	Role requirements, weights, pinned vectors.	Canonical role/skill source and version drift checks.
resource catalogs	Verified study links.	Unique skill/url index; exact normalized matching.
jobs	Portal postings and search projections.	Normalized skills/city; creator derived server-side.
applications	Student/job relationship and status history.	Unique applicant/job compound index.
wishlists / notifications	Saved jobs and user-facing status events.	User-scoped queries and update routes.

Deployment Architecture

The current deployed system uses a free-tier, no-card arrangement: the React/Vite frontend is hosted on Cloudflare Pages, the Express backend is deployed as a Vercel serverless API, and MongoDB Atlas provides persistence. Groq, Gemini, and GitHub are external provider dependencies called only by the backend.



15.1 Production URLs

Service	URL	Purpose
Frontend	https://vortex-6g7.pages.dev	Student and administrator web application.
Backend API	https://vortex-api-eta.vercel.app/api	Authenticated REST API and AI workflows.
Health check	https://vortex-api-eta.vercel.app/api/health	Public deployment and API health verification.

Operational caveat: Vercel serverless functions can cold-start. The frontend sends a lightweight health request on application load, and the dashboard fetches independent data concurrently to reduce perceived wait time.

Deployment Guide

16.1 Prerequisites

- Node.js 22.12 or later and npm.
- GitHub repository access.
- MongoDB Atlas project and database user.
- Groq API key for text generation.
- Gemini API key for embeddings.
- Vercel account for the backend.
- Cloudflare account and Pages project for the frontend.
- Optional GitHub token for higher public API limits.

16.2 Local Verification Before Deployment

```
# From repository root
npm run setup
npm run install:all
docker compose up -d mongo
npm --prefix backend run seed
npm run dev

# In a second shell, verify quality gates
npm --prefix backend test
npm --prefix frontend test
npm --prefix frontend run lint
npm --prefix frontend run build
```

Open <http://localhost:5173>, register or use a development account, confirm login, jobs, applications, New Analysis, report generation, roadmap toggles, and the Assistant before deploying.

16.3 Create MongoDB Atlas

1. Create a free Atlas project and cluster.
2. Create a database user with a strong password.
3. Allow the backend deployment to connect through Network Access. For a production system, prefer a restricted IP strategy; a broad allowlist is only suitable for a disposable demonstration.
4. Create the database name `placement_skill_gap` through the connection string.
5. Store the URI only in a deployment secret; never commit it or place it in a `VITE_` variable.

Deployment Guide: Backend

16.4 Configure Vercel

- 1. Create or link a Vercel project for the backend directory.
- 2. Confirm the serverless entry is `backend/api/index.js` and the project configuration is compatible with the current Vercel function setup.
- 3. Add environment variables in Vercel. Use secret values through the Vercel dashboard or CLI secret mechanism.

Variable	Purpose	Example shape
MONGO_URI or MONGODB_URI	Atlas connection string.	mongodb+srv://.../placement_skill_gap
JWT_SECRET	JWT signing secret.	Long random value
CLIENT_URL	Allowed frontend origins.	Cloudflare Pages URL(s), comma-separated
GROQ_API_KEY	Text extraction, enrichment, assistant.	Secret
GROQ_MODEL	Primary text model.	Configured Groq model name
GEMINI_API_KEY	Embedding provider.	Secret
EMBEDDING_MODEL	Embedding model pin.	gemini-embedding-2
EMBEDDING_VERSION	Vector compatibility baseline.	2026-09
AI_TEXT_PROVIDER	Text provider selector.	groq
AI_EMBEDDING_PROVIDER	Embedding provider selector.	gemini

16.5 Deploy and Verify the Backend

```
# Do not place the token in source control or the report.
cd backend
vercel deploy --prod --yes --token "$VERCEL_TOKEN"

# Public health check
curl https://YOUR-API-DOMAIN/api/health

# Expected shape
{"status":"ok","timestamp":"..."}
```

After deployment, verify login, roles, job listing, a student-owned profile/report, the roadmap endpoint, and an admin-only endpoint. A successful health response proves the function is reachable; it does not by itself prove that MongoDB or AI credentials are valid.

Deployment safety: inspect `.vercelignore` and confirm that `.env`, `.env.render`, local resume storage, and secret files are not bundled.

Deployment Guide: Frontend

16.6 Build with the API URL

The frontend reads its API base URL from `VITE_API_URL`. Only public URLs belong in Vite variables. Never put MongoDB, JWT, Groq, Gemini, or GitHub secrets in the frontend environment.

```
cd frontend
VITE_API_URL="https://YOUR-API-DOMAIN/api" npm run build

# Upload the generated dist directory to Cloudflare Pages
wrangler pages deploy dist \
  --project-name=vortex \
  --branch=main \
  --commit-dirty=true
```

16.7 Configure CORS

1. Copy the production Cloudflare Pages URL.
2. Set the backend `CLIENT_URL` to the exact origin, including any stable deployment alias that must be allowed.
3. Redeploy the backend after changing CORS configuration.
4. Open the frontend and confirm that login and authenticated API requests succeed in the browser.

16.8 Seed or Refresh Data Safely

```
# Run against the intended database, not the local .env by accident.
MONGO_URI="mongodb+srv://.../placement_skill_gap" \
  npm --prefix backend run seed

MONGO_URI="mongodb+srv://.../placement_skill_gap" \
  npm --prefix backend run seed:resources
```

Environment precedence matters: the backend resolves `MONGO_URI` before `MONGODB_URI`. When seeding Atlas from a machine that has a local backend/.env, set `MONGO_URI` explicitly for the command so data is not accidentally written to local MongoDB.

16.9 Smoke Test Checklist

- [] Frontend opens at the public Pages URL.
- [] `/api/health` returns HTTP 200.
- [] Student login and admin login work.
- [] Student can list jobs, save a job, apply, and see notifications.
- [] New Analysis extracts skills and creates a completed report.
- [] Dashboard roadmap endpoint returns the student's own role plans.
- [] Admin can manage jobs and transition application status.
- [] Student cannot access admin routes or another user's report.

Deployment Troubleshooting

Symptom	Likely cause	Action
Health works, login fails	Mongo URI, JWT secret, or database network access.	Inspect Vercel runtime logs and verify Atlas access without exposing secrets.
Browser CORS error	Cloudflare origin missing from <code>CLIENT_URL</code> .	Add the exact origin, redeploy backend, and retry from a clean browser session.
Jobs load but analysis fails	Groq/Gemini variables, quota, model mismatch, or ontology not seeded.	Check AI configuration, provider quota, embedding model/version, and resource/ontology seeds.
Analysis is slow on first request	Serverless cold start or provider latency.	Use the health warm-up, wait for the bounded job status, and inspect lifecycle logs.
Roadmap is empty	No completed report for the user/role, or report has no gaps.	Run New Analysis and confirm <code>GET /api/report/roadmap</code> with the same JWT.
Seeded data missing from Atlas	Local <code>MONGO_URI</code> overrode the exported Atlas URI.	Set <code>MONGO_URI</code> explicitly for the seed command and verify the target database.
Resume upload rejected	Not a real PDF, too large, or serverless upload cap.	Use a valid PDF within the configured limit; do not bypass magic-byte validation.
Vercel deploy configuration error	Invalid function config or bundled environment files.	Keep the current valid <code>vercel.json</code> ; ensure ignored files stay out of the bundle.

Rollback process: keep the previous known-good deployment URL and use the hosting provider's deployment history to promote the previous build. Do not edit production secrets to compensate for an application code regression.

16.10 Operational Boundaries

- Vercel temporary filesystem storage is not durable; persistent resume storage should be moved to object storage for production.
- Free-tier AI providers have quotas and transient overload; provider errors are mapped to controlled API responses.
- Cold starts and external provider latency mean a first request can be slower than a warm request.
- Atlas Network Access and demo credentials must be hardened before any non-academic use.

Testing and Verification

17.1 Automated Test Coverage

Area	Coverage examples	Current result
Authentication	Registration, login, hashing, JWT, invalid credentials, expiry behavior.	Passed
Upload security	Missing file, spoofed extension, magic bytes, oversized PDF, ownership, delete cleanup.	Passed
AI extraction	Structured schema, merging, confidence rules, provider errors, graceful public-profile failure.	Passed
Scoring	Formula, thresholds, gap priority, exact normalized resources, finite score guard.	Passed
Analysis jobs	Idempotency, cooldown, status polling, failure path, cached embedding compatibility.	Passed
Jobs	CRUD, RBAC, filters, normalization, pagination, invalid ids, empty results.	Passed
Applications	Duplicate prevention, ownership, status transitions, history, admin summary.	Passed
Roadmaps	Latest report per role, resource hydration, own-only access, empty state.	Passed
Frontend	Routes, auth views, dashboard, roadmap selector/collapse, upload form, assistant answer, theme.	Passed

17.2 Latest Quality Gate

154 Backend tests passed	2 Opt-in backend tests skipped	41 Frontend tests passed	0 Frontend lint errors
--	--	--	--

The production frontend build completed successfully. The current expansion was committed as 09788b3 and pushed to the main branch. Live smoke checks confirmed 21 roles, 180 keyword names, new-role job filters, specific video resources, a new Frontend Developer analysis, API health, roadmap status handling, and cached embeddings.

17.3 Manual Verification

- Student and administrator login verified against the deployed system.
- Job listing, filters, applications, wishlist, notifications, and admin pipeline verified.
- New Analysis produced extracted skills, readiness score, study items, and curated resource links.
- Per-role roadmap returned the latest completed report for each role and preserved completion toggles.
- Candidate skill embedding cache persisted model, version, and vector dimensions.
- Non-resume PDFs were rejected with 422 not_a_resume; converter-generated resume PDFs were accepted and extracted.
- New role analysis completed for Frontend Developer with documentation and video resources in the plan.

Security, Limitations, and Future Work

18.1 Current Limitations

- Readiness scores are guidance signals, not proof of ability or a guarantee of employment.
- Public GitHub and LeetCode data can be unavailable, stale, rate-limited, or incomplete.
- AI output quality depends on provider availability, prompt constraints, and source evidence quality.
- Vercel temporary storage is not a durable long-term resume store.
- Free-tier provider quotas and serverless cold starts can increase response time.
- Current application review is a workflow tool, not a replacement for human hiring decisions.

18.2 Future Enhancements

1. Move resume files to durable object storage with signed, time-limited download URLs.
2. Add background job infrastructure for resilient analysis retries and larger workloads.
3. Add model drift dashboards and scheduled ontology/resource refresh audits.
4. Support institution-level placement analytics with privacy-preserving aggregation.
5. Add calendar-based study scheduling, reminders, and milestone analytics.
6. Add richer interview preparation linked to verified gaps and application role.
7. Introduce automated CI for tests, lint, build, dependency audits, and deployment previews.
8. Improve internationalization, accessibility audits, and screen-reader regression coverage.

18.3 Responsible AI Position

Responsible use: Vortex should help a student understand and act on evidence. It should not fabricate achievements, rank people as objectively employable, expose private data, or replace a qualified human decision-maker.

Conclusion and Viva Summary

Vortex demonstrates how a conventional job board can be extended into an evidence-based preparation system. Its value comes from the connection between the application lifecycle and the learning lifecycle: a student can discover a role, apply, inspect readiness for that role, follow a roadmap, and receive grounded guidance without leaving the same workspace.

The implementation keeps responsibilities separated. React presents the workflow, Express protects and coordinates it, MongoDB stores durable state, Groq structures and explains text, Gemini provides pinned vector representations, and deterministic application code controls scoring, thresholds, ownership, and verified resources. This design makes the system suitable for academic demonstration while also exposing clear paths for production hardening.

Jobs + Evidence + Role Ontology + Deterministic Scoring + Grounded Guidance =
Actionable Career Preparation

One-sentence Explanation

Vortex stores job postings and candidate profiles, extracts evidence-based technical skills from resumes and public coding profiles, compares them with role requirements, calculates readiness, identifies skill gaps, and provides personalized preparation through a verified study roadmap and grounded AI Assistant.

Viva Questions to Prepare

1. Why is public registration restricted to the student role?
2. Why should the frontend never decide ownership or admin access?
3. What is the difference between extraction, matching, scoring, and explanation?
4. Why are embeddings cached and why is model/version metadata stored?
5. How does the system prevent React from satisfying a PyTorch requirement?
6. What happens if GitHub, Groq, or Gemini is unavailable?
7. How does the compound application index prevent duplicates?
8. How would you make resume storage durable beyond a serverless filesystem?

End of main report. Appendices provide the repository map, example requests, and a deployment handover checklist.

Repository Structure

```
pride_proj/
├── backend/
│   ├── api/                # Vercel serverless entry
│   ├── src/
│   │   ├── config/         # env, database, CORS, roles
│   │   ├── controllers/    # request handlers
│   │   ├── middleware/     # auth, upload, ownership, errors
│   │   ├── models/         # Mongoose schemas
│   │   ├── routes/         # REST route modules
│   │   └── services/       # AI, extraction, scoring, storage
│   ├── ontology/          # role skill matrices
│   ├── resources/         # verified docs, courses, practice, and videos
│   ├── scripts/           # seed, admin, server, audits
│   ├── tests/             # Vitest + Supertest
│   └── vercel.json         # serverless function configuration
├── frontend/
│   ├── src/
│   │   ├── components/     # reusable UI and controls
│   │   ├── hooks/          # shared keyword/catalog hooks
│   │   ├── context/        # auth and theme state
│   │   ├── pages/          # route-level screens
│   │   ├── test/           # frontend component tests
│   │   └── api/            # Axios client
│   ├── public/            # redirects and static assets
│   └── vite.config.js
├── docs/                  # API, schema, setup, reports
├── scripts/               # root setup and dev launchers
├── docker-compose.yml     # local MongoDB
└── README.md
```

Appendix A.1 Important Source Files

File	Responsibility
backend/src/app.js	Express app, middleware, health route, router mounting.
backend/src/services/analysisService.js	Analysis lifecycle, cache reuse, ontology checks, report persistence.
backend/src/services/scoringService.js	Matching, score, categories, priority, study-plan base.
backend/src/services/skillService.js	Resume/profile extraction, merging, confidence, embedding cache.
backend/src/controllers/skill.controller.js	Cached canonical keyword catalog for filters and job forms.
frontend/src/pages/DashboardPage.jsx	Score, role selector, collapsible roadmap, application preparation, wishlist.
frontend/src/components/StudyPlan.jsx	Accessible collapsible roadmap card and completion toggles.
frontend/src/components/SkillMultiSelect.jsx	Searchable multi-select keyword picker for job search and admin job creation.
frontend/src/pages/UploadPage.jsx	Upload, extraction review, analyze, polling, dashboard handoff.

Representative API Examples

B.1 Login

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "student@example.com",
  "password": "your-password"
}

200 OK
{
  "token": "<JWT>",
  "user": { "id": "...", "name": "Student", "role": "student" }
}
```

B.2 Roadmap

```
GET /api/report/roadmap
Authorization: Bearer <JWT>

200 OK
{
  "roadmaps": [{
    "target_role": "AI Engineer",
    "report_id": "...",
    "status": "completed",
    "score": 21,
    "gap_count": 12,
    "study_plan": [{
      "skill": "PyTorch",
      "priority": 1,
      "done": false,
      "resources": [{ "title": "PyTorch Tutorials", "type": "documentation", "verified": true }]
    }]
  }]
}
```

B.3 Skills for Job Filters

```
GET /api/skills
Authorization: Bearer <JWT>

200 OK
{
  "skills": ["AI Safety", "AWS", "Flutter", "React", "Solidity", "..."]
}
```

B.4 Toggle Roadmap Item

```
PATCH /api/report/<reportId>/study-plan/<itemId>
Authorization: Bearer <JWT>

200 OK
{ "report_id": "...", "item_id": "...", "done": true }
```

B.5 Create Job as Administrator

```
POST /api/jobs
Authorization: Bearer <ADMIN_JWT>
Content-Type: application/json

{
  "title": "Backend Engineer",
  "company": "Example Labs",
  "skills": ["Node.js", "Express", "MongoDB"],
  "experienceLevel": 1,
  "city": "Chennai",
  "description": "Build reliable API services."
}
```

B.6 Common Status Codes

Status	Meaning in Vortex
200	Successful read, update, or health response.
201	Resource created, such as a user, profile, job, or application.
400	Malformed request or validation failure.
401 / 403	Missing/invalid authentication or insufficient permissions.
409 / 429	Business conflict, active report, cooldown, or rate limit.

All examples use placeholders. Real tokens, passwords, connection strings, and provider keys must be supplied through secure environment configuration.

Deployment Handover Checklist

- [] Repository is on the intended branch and the working tree is clean.
- [] Backend environment variables are configured in Vercel, not committed.
- [] Atlas database user, database name, and network access are confirmed.
- [] Backend CORS allows the exact Cloudflare Pages origin.
- [] Vercel function entry, max duration, and ignore rules are valid.
- [] Ontology and verified resource catalog are seeded in the intended database.
- [] Frontend VITE_API_URL points to the deployed API base path.
- [] Cloudflare Pages deployment completes and SPA redirects are present.
- [] Health endpoint returns HTTP 200.
- [] Student login, admin login, job listing, application, and notification flows work.
- [] New Analysis completes and the dashboard roadmap appears.
- [] Roadmap role selector and collapse/expand behavior work on desktop and mobile.
- [] Admin application status transition produces a student notification.
- [] Cross-user report and admin-route authorization checks return 403.
- [] Production logs contain no passwords, tokens, resume text, or API keys.
- [] Previous deployment is retained for rollback.

C.1 Final Verification Commands

```
# Backend
npm --prefix backend test

# Frontend
npm --prefix frontend test
npm --prefix frontend run lint
VITE_API_URL="https://YOUR-API-DOMAIN/api" npm --prefix frontend run build

# Live health
curl -i https://YOUR-API-DOMAIN/api/health
```

Handover result: a future maintainer should be able to identify the two deployed services, configure secrets without committing them, seed the intended database, run the quality gates, smoke-test both roles, and roll back safely.